Project 8: Simple Gradle Project – Creating and Running a Gradle Java Application

Objective

The goal of this project is to create a simple Gradle-based Java application from scratch, understand the project structure, configure the build file, build and run the application, and create a custom Gradle task.

Tools Used

- Gradle Build Tool
- Java JDK
- Ubuntu Terminal
- Text Editor (gedit / nano / VS Code)

Step 1: Create a Project Directory

Open the terminal and run the following command to create a new folder for the Gradle project:

```
mkdir MyGradleApp
cd MyGradleApp
```

This creates the project directory and navigates into it. All subsequent Gradle commands will be run from inside this folder.

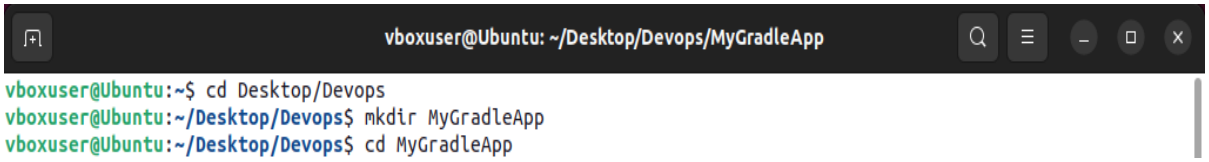

Fig 1: Terminal showing mkdir MyGradleApp, cd MyGradleApp, and initial gradle init output

Step 2: Initialize the Gradle Project

Run the following command to initialise a new Gradle Java application project:

```
gradle init --type java-application
```

Gradle will generate the standard project structure with a sample App.java, AppTest.java, build.gradle, and settings.gradle files. A BUILD SUCCESSFUL message confirms that initialisation completed without errors.

```
vboxuser@Ubuntu:~/Desktop/Devops/MyGradleApp$ gradle init --type java-application


BUILD SUCCESSFUL in 0s
2 actionable tasks: 2 executed
vboxuser@Ubuntu:~/Desktop/Devops/MyGradleApp$
```

Fig 2: gradle init --type java-application output showing BUILD SUCCESSFUL

Step 3: List Project Files

After initialisation, list the files generated by Gradle using:

```
ls
```

The following files and folders are created:

- build.gradle – the main build configuration file
- settings.gradle – project settings file
- gradle/ – folder containing the Gradle wrapper
- gradlew / gradlew.bat – Gradle wrapper scripts for Linux and Windows
- src/ – source code directory

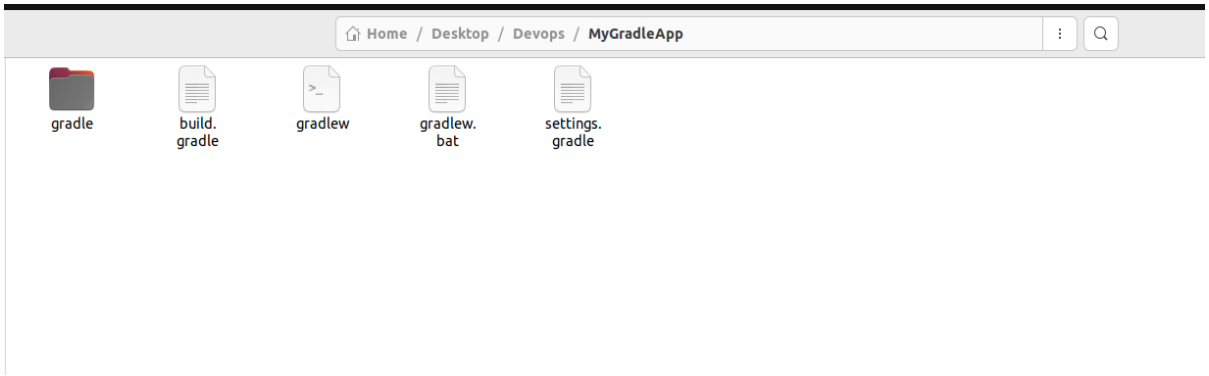

Fig 3: File manager view of MyGradleApp showing generated project files

Step 4: View Project Structure

To see the full folder hierarchy, run:

```
tree
```

The project follows a standard Gradle/Maven directory layout:

- src/main/java – contains App.java (main application source)
- src/test/java – contains AppTest.java (unit test source)
- gradle/wrapper – contains wrapper JAR and properties for Gradle version management

```
vboxuser@Ubuntu:~/Desktop/Devops/MyGradleApp$ tree
.
├── build.gradle
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle
└── src
    ├── main
    │   └── java
    │       └── App.java
    └── test
        └── java
            └── AppTest.java

7 directories, 8 files
vboxuser@Ubuntu:~/Desktop/Devops/MyGradleApp$
```

Fig 4: tree command output showing the full Gradle project directory structure

Step 5: Edit build.gradle

Open the build.gradle file using any text editor:

```
gedit build.gradle
```

The build.gradle file is the heart of every Gradle project. It defines the plugins, repositories, dependencies, and application entry point. Key sections include:

- apply plugin: 'java' – adds Java compilation support
- apply plugin: 'application' – enables the run task
- repositories { jcenter() } – declares where to fetch dependencies
- dependencies { ... } – lists libraries like Guava and JUnit
- mainClassName = 'App' – specifies the main class to run

```
Open ▾                    *build.gradle                                Save
                  ~/Desktop/Devops/MyGradleApp
1 /*
2 * This build file was generated by the Gradle 'init' task.
3 *
4 * This generated file contains a sample Java project to get you started.
5 * For more details take a look at the Java Quickstart chapter in the Gradle
6 * user guide available at https://docs.gradle.org/4.4.1/userguide/tutorial_java_projects.html
7 */
8
9 // Apply the java plugin to add support for Java
10 apply plugin: 'java'
11
12 // Apply the application plugin to add support for building an application
13 apply plugin: 'application'
14
15 // In this section you declare where to find the dependencies of your project
16 repositories {
17     // Use jcenter for resolving your dependencies.
18     // You can declare any Maven/Ivy/file repository here.
19     jcenter()
20 }
21
22 dependencies {
23     // This dependency is found on compile classpath of this component and consumers.
24     compile 'com.google.guava:guava:23.0'
25
26     // Use JUnit test framework
27     testCompile 'junit:junit:4.12'
28 }
29
30 // Define the main class for the application
31 mainClassName = 'App'
32
33
34
```

Fig 5: build.gradle file opened in editor showing plugins, repositories, dependencies and mainClassName

Step 6: Build the Project

Compile and build the entire project by running:

```
gradle build
```

Gradle will download all declared dependencies (such as Guava and JUnit from jcenter), compile the source code, run the tests, and package the application. A BUILD SUCCESSFUL message with the number of actionable tasks confirms a successful build.

```
7 directories, 8 files
vboxuser@Ubuntu:~/Desktop/Devops/MyGradleApp$ gradle build
Download https://jcenter.bintray.com/com/google/guava/guava/23.0/guava-23.0.pom
Download https://jcenter.bintray.com/com/google/guava/guava-parent/23.0/guava-parent-23.0.pom
Download https://jcenter.bintray.com/com/google/errorprone/error_prone_annotations/2.0.18/error_prone_annotation
s-2.0.18.pom
Download https://jcenter.bintray.com/com/google/errorprone/error_prone_parent/2.0.18/error_prone_parent-2.0.18.p
om
Download https://jcenter.bintray.com/org/codehaus/mojo/animal-sniffer-annotations/1.14/animal-sniffer-annotation
s-1.14.pom
Download https://jcenter.bintray.com/com/google/code/findbugs/jsr305/1.3.9/jsr305-1.3.9.pom
Download https://jcenter.bintray.com/com/google/j2objc/j2objc-annotations/1.1/j2objc-annotations-1.1.pom
Download https://jcenter.bintray.com/org/codehaus/mojo/animal-sniffer-parent/1.14/animal-sniffer-parent-1.14.pom
Download https://jcenter.bintray.com/org/codehaus/mojo/mojo-parent/34/mojo-parent-34.pom
Download https://jcenter.bintray.com/org/codehaus/codehaus-parent/4/codehaus-parent-4.pom
Download https://jcenter.bintray.com/com/google/j2objc/j2objc-annotations/1.1/j2objc-annotations-1.1.jar
Download https://jcenter.bintray.com/com/google/errorprone/error_prone_annotations/2.0.18/error_prone_annotation
s-2.0.18.jar
Download https://jcenter.bintray.com/com/google/guava/guava/23.0/guava-23.0.jar
Download https://jcenter.bintray.com/org/codehaus/mojo/animal-sniffer-annotations/1.14/animal-sniffer-annotation
s-1.14.jar
Download https://jcenter.bintray.com/com/google/code/findbugs/jsr305/1.3.9/jsr305-1.3.9.jar

BUILD SUCCESSFUL in 18s
7 actionable tasks: 7 executed
vboxuser@Ubuntu:~/Desktop/Devops/MyGradleApp$
```

Fig 6: gradle build output showing dependency downloads and BUILD SUCCESSFUL in 18s

Step 7: Run the Application

Run the application using the Gradle run task:

```
gradle run
```

Gradle executes the main class (App.java) and displays the program output. The default generated App.java prints "Hello world." to the terminal. A BUILD SUCCESSFUL message confirms the run completed successfully.

```
vboxuser@Ubuntu:~/Desktop/Devops/MyGradleApp$ gradle run

> Task :run
Hello world.


BUILD SUCCESSFUL in 0s
2 actionable tasks: 1 executed, 1 up-to-date
vboxuser@Ubuntu:~/Desktop/Devops/MyGradleApp$
```

Fig 7: gradle run output showing "> Task :run" and "Hello world." followed by BUILD SUCCESSFUL

Step 8: Add a Custom Task to build.gradle

Open build.gradle again and add the following custom task at the end of the file:

```
task display {
    doLast {
        println "6D devops class"
    }
}
```

This defines a custom Gradle task named display. When executed, it runs the doLast block and prints the specified message to the terminal. Save the file after adding the task.

Step 9: Run the Custom Task

Execute the custom task using:

```
gradle display
```

Gradle locates the display task in build.gradle and runs it. The terminal shows "> Task :display" followed by the custom message "6D devops class". A BUILD SUCCESSFUL message with 1 actionable task confirms successful execution.

```
vboxuser@Ubuntu:~/Desktop/Devops/MyGradleApp$ gradle display

> Task :display
6D devops class


BUILD SUCCESSFUL in 0s
1 actionable task: 1 executed
vboxuser@Ubuntu:~/Desktop/Devops/MyGradleApp$
```

Fig 8: gradle display output showing "> Task :display", "6D devops class" and BUILD SUCCESSFUL